

Data Visualisation

Matplotlib & Seaborn

Practical Session 3 — Lab Sheet

B.Tech (Computer Engineering) — III Year
Machine Learning Laboratory

July 6, 2026

Instructions for Students: Work through each topic in order. Read the *Concept* box, run the *Example Code* in a Jupyter Notebook / Google Colab, verify the plot appears, then solve the *Exercise* underneath before moving on. Save all code in a single notebook named `lab3_yourrollno.ipynb`. This sheet uses only the standard Iris and Wine datasets from Sessions 1 & 2.

Contents

Why Visualise Before Modelling?	2
1 Part I — Matplotlib	3
1.1 Figure Anatomy and the Object-Oriented API	3
1.2 Line Plots	3
1.3 Scatter Plots	4
1.4 Histograms	5
1.5 Bar Charts	5
1.6 Box Plots	6
1.7 Multi-Panel Subplots	7
2 Part II — Seaborn	8
2.1 Scatter Plot with Hue (<code>scatterplot</code>)	8
2.2 Distribution Plots (<code>histplot</code> & <code>kdeplot</code>)	8
2.3 Box Plots and Violin Plots	9
2.4 Pair Plots (<code>pairplot</code>)	10
2.5 Heatmaps (<code>heatmap</code>)	10
2.6 Bar and Count Plots (<code>barplot</code> & <code>countplot</code>)	11
2.7 FacetGrid — Small Multiples	12
3 Capstone Exercise — A Complete Exploratory Visual Analysis	13
Appendix: Quick Reference Cheat Sheet	14

Why Visualise Before Modelling?

Numbers in a table are opaque. An outlier, a bimodal distribution, a near-perfect linear relationship between two features, or a class imbalance that will silently wreck your accuracy – all of these are invisible until you plot the data. Data visualisation is therefore **not a presentation tool tacked on at the end**; it is a core diagnostic step done *before* any model is trained and again *after* predictions are made (to inspect errors and calibration).

This session covers two complementary libraries. **Matplotlib** is the low-level engine: it gives you total control over every pixel of a figure. **Seaborn** is a high-level statistical graphics layer built *on top of* Matplotlib: with a single function call it produces publication-quality plots that would take dozens of Matplotlib lines to build by hand. You will use both – Matplotlib when you need fine-grained control or want to understand what is happening under the hood, and Seaborn when you need to explore data quickly.

Notebook setup. Put the following imports at the top of your notebook and run them once before anything else.

Example Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris, load_wine

# load datasets into DataFrames
iris = load_iris()
iris_df = pd.DataFrame(iris.data, columns=iris.feature_names)
iris_df['species'] = [iris.target_names[t] for t in iris.target]

wine = load_wine()
wine_df = pd.DataFrame(wine.data, columns=wine.feature_names)
wine_df['wine_class'] = [wine.target_names[t] for t in wine.target]

sns.set_theme(style='whitegrid') # apply a clean seaborn theme globally
```

Throughout this sheet, **Iris** is used in every *Example Code* block and **Wine** is used in every *Exercise*.

1 Part I — Matplotlib

1.1 Figure Anatomy and the Object-Oriented API

Concept

Every Matplotlib plot lives inside a **Figure** (the whole canvas). A Figure contains one or more **Axes** objects (each Axes is one individual plot panel). The recommended workflow is the **object-oriented (OO) API**: call `plt.subplots()` to get a `(fig, ax)` pair, then call methods on `ax` to draw on it. This is clearer and more reusable than the older `plt.plot()` “state-machine” style. Key anatomy: `ax.set_title()`, `ax.set_xlabel()`, `ax.set_ylabel()`, `ax.legend()`, and `ax.set_xlim()` / `ax.set_ylim()` control every label and axis range. The figure is saved with `plt.savefig()` or shown interactively with `plt.show()`.

Example Code

```
fig, ax = plt.subplots(figsize=(6, 3)) # Figure with one Axes panel

ax.plot([1, 2, 3, 4], [4, 1, 3, 5],
        color='steelblue', linewidth=2, marker='o', label='signal')
ax.set_title('Anatomy of a Matplotlib figure')
ax.set_xlabel('x axis label')
ax.set_ylabel('y axis label')
ax.legend()
plt.tight_layout()
plt.show()
```

Exercise

1. Create a Figure of size (7, 4) with a single Axes. Plot the curve $y = x^2$ for $x \in [-3, 3]$ using 50 points from `np.linspace()`.
2. Add a descriptive title, x-label, y-label, and a legend entry `'$y = x^2$'`.
3. Save the figure to a file called `parabola.png` using `plt.savefig('parabola.png', dpi=150, bbox_inches='tight')` and confirm it appears in your file browser.

1.2 Line Plots

Concept

Line plots show **trends** – how a quantity changes continuously across an ordered axis (time, sample index, epoch number). Calling `ax.plot()` multiple times on the same `ax` overlays multiple lines; the `label` argument and a call to `ax.legend()` keep them distinguishable. The `alpha` parameter (0–1) controls transparency, useful when lines overlap.

Example Code

```
fig, ax = plt.subplots(figsize=(8, 3))

colors = ['steelblue', 'tomato', 'seagreen']
for species, grp in iris_df.groupby('species'):
    ax.plot(grp.index, grp['sepal length (cm)'],
            color=colors[list(iris_df['species'].unique()).index(species)],
            alpha=0.7, label=species)
```

```
ax.set_xlabel('Sample index')
ax.set_ylabel('Sepal length (cm)')
ax.set_title('Sepal length across samples, coloured by species')
ax.legend()
plt.tight_layout()
plt.show()
```

Exercise

1. Using the Wine DataFrame, plot `alcohol` values (sample index on x) as three separate lines, one per `wine_class`, each a different colour.
2. Add a dotted horizontal line (`ax.axhline()`) at the overall mean alcohol level to show which classes sit above/below average.
3. Change the line style of the second class to dashed (`linestyle='--'`) and the third to dash-dot (`linestyle='-.'`). Check the Matplotlib *linestyle* documentation in-notebook with `help(ax.plot)`.

1.3 Scatter Plots

Concept

Scatter plots reveal **relationships** between two continuous features. When samples are colour-coded by class label (`c=` colour list or loop), you can immediately see whether the two features separate the classes – which is exactly the question you ask before choosing a classifier. Use `ax.scatter()` for full control or Seaborn's `scatterplot` (covered in Part II) for automatic legend handling.

Example Code

```
colors = ['steelblue', 'tomato', 'seagreen']
fig, ax = plt.subplots(figsize=(5, 4))

for i, species in enumerate(iris.target_names):
    mask = iris_df['species'] == species
    ax.scatter(iris_df.loc[mask, 'petal length (cm)'],
               iris_df.loc[mask, 'petal width (cm)'],
               color=colors[i], label=species, alpha=0.75, edgecolors='white')

ax.set_xlabel('Petal length (cm)')
ax.set_ylabel('Petal width (cm)')
ax.set_title('Petal dimensions coloured by species')
ax.legend()
plt.tight_layout()
plt.show()
```

Exercise

1. Create a scatter plot of `alcohol` (x) vs. `color_intensity` (y) for the Wine dataset, with each `wine_class` in a different colour.
2. Adjust marker size with the `s=` parameter (try `s=60`) and add `edgecolors='white'` to make the points pop.
3. Create a second scatter plot comparing any two features of your choice. Which pair looks

more linearly separable by class? Write your answer as a comment.

1.4 Histograms

Concept

A histogram shows the **distribution** of a single continuous variable. Overlapping, semi-transparent histograms per class (using `alpha < 1`) reveal whether feature distributions differ across classes – if they do, that feature will be useful for a classifier. The `bins` argument controls resolution: too few bins hide structure; too many bins add noise.

Example Code

```
colors = ['steelblue', 'tomato', 'seagreen']
fig, ax = plt.subplots(figsize=(6, 3))

for i, species in enumerate(iris.target_names):
    data = iris_df.loc[iris_df['species'] == species, 'petal length (cm)']
    ax.hist(data, bins=15, color=colors[i], alpha=0.6,
            label=species, edgecolor='white')

ax.set_xlabel('Petal length (cm)')
ax.set_ylabel('Count')
ax.set_title('Distribution of petal length per species')
ax.legend()
plt.tight_layout()
plt.show()
```

Exercise

1. Plot overlapping histograms of `proline` for all three Wine classes. Which class has the highest `proline` values?
2. Change `density=True` in your `ax.hist()` call to normalise the y-axis to probability density instead of raw counts. Does the visual conclusion change?
3. Add a vertical dashed line (`ax.axvline()`) at the mean of each class's `proline` distribution, in the matching class colour.

1.5 Bar Charts

Concept

Bar charts compare a numeric quantity **across categories**. For ML, the most common uses are: comparing a per-class mean of a feature (to see which features discriminate classes), and comparing per-class sample counts (to spot class imbalance before training). `ax.bar()` draws the bars; `yerr=` adds error bars; and `np.arange(n)` paired with `ax.set_xticks()` and `ax.set_xticklabels()` positions and labels them correctly.

Example Code

```
means = iris_df.groupby('species')['petal length (cm)'].mean()
stds = iris_df.groupby('species')['petal length (cm)'].std()
colors = ['steelblue', 'tomato', 'seagreen']

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
```

```
# left: mean + error bars
x = np.arange(len(means))
axes[0].bar(x, means, yerr=stds, capsize=5,
            color=colors, edgecolor='black', alpha=0.8)
axes[0].set_xticks(x)
axes[0].set_xticklabels(means.index)
axes[0].set_ylabel('Mean petal length (cm)')
axes[0].set_title('Mean  $\pm$  1 SD per species')

# right: sample counts (class balance check)
counts = iris_df['species'].value_counts()
axes[1].bar(counts.index, counts.values, color=colors, edgecolor='black', alpha=0.8)
axes[1].set_ylabel('Count')
axes[1].set_title('Sample count per species')

plt.tight_layout()
plt.show()
```

Exercise

1. For the Wine dataset, create a bar chart of the mean `alcohol` per class with error bars showing ± 1 standard deviation.
2. Create a second bar chart showing the sample *count* per `wine_class`. Is the dataset balanced?
3. Produce a **grouped bar chart** comparing the mean `flavanoids` and mean `total_phenols` side-by-side for each class. Hint: offset two bar groups by shifting `x` by ± 0.2 and add a legend.

1.6 Box Plots

Concept

A box plot compactly shows a feature's **distribution and outliers** in one panel: the box spans the IQR ($Q1-Q3$), the line inside is the median, the whiskers extend to $1.5 \times IQR$, and individual points beyond the whiskers are drawn as fliers (potential outliers). Plotting one box per class side-by-side makes class differences immediately apparent – a key step before training any model that is sensitive to feature scales.

Example Code

```
data_by_species = [
    iris_df.loc[iris_df['species'] == sp, 'petal length (cm)'].values
    for sp in iris.target_names
]
fig, ax = plt.subplots(figsize=(5, 4))
bp = ax.boxplot(data_by_species,
                tick_labels=iris.target_names, patch_artist=True)

colors = ['steelblue', 'tomato', 'seagreen']
for patch, c in zip(bp['boxes'], colors):
    patch.set_facecolor(c)
    patch.set_alpha(0.7)

ax.set_ylabel('Petal length (cm)')
ax.set_title('Petal length distribution per species')
plt.tight_layout()
```

```
plt.show()
```

Exercise

1. Create a box plot of `color_intensity` for the three Wine classes with each box a different colour.
2. Repeat for `hue`. Which feature shows clearer class separation?
3. Create a (1, 4) subplot grid and draw one box plot per numeric feature of your choice across the three classes, giving a four-panel overview.

1.7 Multi-Panel Subplots

Concept

`plt.subplots(nrows, ncols)` returns a 2-D array of `Axes` objects; `axes.ravel()` flattens it so you can loop over panels easily. `fig.suptitle()` adds a shared title above all panels. `plt.tight_layout()` automatically adjusts spacing to prevent overlapping labels. Multi-panel layouts are the standard way to give a full feature-by-feature overview of a dataset in one figure.

Example Code

```
features = iris.feature_names
fig, axes = plt.subplots(2, 2, figsize=(9, 6))

colors = ['steelblue', 'tomato', 'seagreen']
for ax, feat in zip(axes.ravel(), features):
    for i, sp in enumerate(iris.target_names):
        data = iris_df.loc[iris_df['species'] == sp, feat]
        ax.hist(data, bins=12, color=colors[i], alpha=0.6, label=sp)
    ax.set_title(feat)
    ax.set_xlabel('cm')

handles, labels = axes[0, 0].get_legend_handles_labels()
fig.legend(handles, labels, loc='lower right', title='Species')
fig.suptitle('All feature distributions by species', fontsize=13)
plt.tight_layout()
plt.show()
```

Exercise

1. Create a (3, 5) subplot grid showing histograms of all 13 numeric Wine features (one per panel), coloured by `wine_class`. Add a shared super-title and a single shared legend.
2. Modify the layout so each panel has a *scatter* plot (alcohol vs. that feature) instead of a histogram.
3. Use `plt.subplots_adjust(hspace=0.5, wspace=0.4)` instead of `tight_layout()` and compare the effect on label spacing.

2 Part II — Seaborn

Seaborn wraps Matplotlib to produce statistical graphics with far less code. It understands Pandas DataFrames natively, handles legends and colour palettes automatically, and integrates well with the familiar (`fig`, `ax`) workflow by accepting an `ax=` argument. The golden rule: **pass the DataFrame and column names as strings**; Seaborn looks up the data itself.

2.1 Scatter Plot with Hue (`scatterplot`)

Concept

`sns.scatterplot()` extends the Matplotlib scatter by automatically assigning colours to each level of a `hue` column and drawing a labelled legend. The `style` parameter further differentiates classes by marker shape – useful when printing in black-and-white. All seaborn axes-level functions accept `ax=` so they slot into any `plt.subplots()` layout.

Example Code

```
fig, ax = plt.subplots(figsize=(5, 4))
sns.scatterplot(data=iris_df,
                x='petal length (cm)', y='petal width (cm)',
                hue='species', style='species',
                palette='tab10', alpha=0.8, ax=ax)
ax.set_title('Petal dimensions by species')
plt.tight_layout()
plt.show()
```

Exercise

1. Create a Seaborn scatter plot of `alcohol` vs. `color_intensity` in the Wine DataFrame, coloured by `wine_class`.
2. Add `style='wine_class'` so each class also gets a different marker shape.
3. Try `palette='Set2'` and `palette='husl'`. Pick the one you find most readable for colourblind-inclusive presentations and explain your choice in one sentence.

2.2 Distribution Plots (`histplot` & `kdeplot`)

Concept

`sns.histplot()` combines a histogram and an optional KDE (Kernel Density Estimate) curve in one call. `sns.kdeplot()` draws *only* the smooth density curve, which is useful for clean overlapping comparisons without the visual noise of bar edges. Setting `fill=True` shades under the curve, and `alpha` controls its transparency.

Example Code

```
fig, axes = plt.subplots(1, 2, figsize=(11, 3.5))

# Histogram with embedded KDE
sns.histplot(data=iris_df, x='sepal length (cm)', hue='species',
             kde=True, bins=15, alpha=0.5, ax=axes[0])
axes[0].set_title('Histogram + KDE overlay')

# KDE only -- cleaner for overlapping comparisons
```

```
sns.kdeplot(data=iris_df, x='sepal length (cm)', hue='species',
            fill=True, alpha=0.3, ax=axes[1])
axes[1].set_title('KDE only')

plt.tight_layout()
plt.show()
```

Exercise

1. Plot the distribution of **proline** for each Wine class using `sns.histplot`. Set `kde=True` and `bins=20`. Which class has the widest spread?
2. Replace with `sns.kdeplot` and set `fill=True`. Is the KDE or the histogram easier to read here? Comment your answer.
3. Create a side-by-side panel with one `histplot` and one `kdeplot` (using the same feature) to directly compare the two representations.

2.3 Box Plots and Violin Plots

Concept

`sns.boxplot()` gives the classic five-number summary (min, Q1, median, Q3, max) per category. `sns.violinplot()` adds a mirrored KDE “violin” shape on each side, showing the full distribution density – it reveals bimodality that a box plot would hide. Use `inner='quartile'` inside a violin to also draw the IQR lines.

Example Code

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

sns.boxplot(data=iris_df, x='species', y='petal length (cm)',
            hue='species', palette='tab10', legend=False, ax=axes[0])
axes[0].set_title('Box plot: petal length by species')

sns.violinplot(data=iris_df, x='species', y='petal length (cm)',
               hue='species', palette='tab10', legend=False,
               inner='quartile', ax=axes[1])
axes[1].set_title('Violin plot: petal length by species')

plt.tight_layout()
plt.show()
```

Exercise

1. Create a side-by-side box vs. violin plot for **color_intensity** across the three Wine classes.
2. Does the violin plot reveal any bimodality in any class that the box plot hides? State your observation as a comment.
3. Use `inner='stick'` in the violin plot to show all individual data points inside the violins instead of quartile lines.

2.4 Pair Plots (pairplot)

Concept

`sns.pairplot()` produces a grid of scatter plots for every pair of numeric features, with distribution plots on the diagonal. It is the fastest way to get a complete picture of all pairwise relationships in a dataset. `hue=` colour-codes each class, `diag_kind='kde'` replaces diagonal histograms with KDE curves, and `plot_kws` passes extra arguments (such as `alpha`) to the off-diagonal scatter plots.

Example Code

```
g = sns.pairplot(iris_df, hue='species', palette='tab10',
                  diag_kind='kde', plot_kws={'alpha': 0.6})
g.figure.suptitle('Pairwise relationships -- Iris dataset', y=1.01)
plt.tight_layout()
plt.show()
```

Exercise

1. Run `sns.pairplot` on the Wine DataFrame with `hue='wine_class'`. Which pair of features shows the cleanest class separation?
2. By default, `pairplot` uses all numeric columns. Use the `vars=` argument to restrict the plot to just `['alcohol', 'color_intensity', 'flavanoids', 'proline']`.
3. Change `diag_kind='hist'` and compare the diagonal panels. Which is more informative for this dataset?

2.5 Heatmaps (heatmap)

Concept

`sns.heatmap()` visualises a 2-D matrix as a colour-coded grid. Its most common ML use is displaying a **correlation matrix**: bright red cells (near +1) highlight redundant or tightly coupled features; dark blue cells (near -1) highlight inversely related ones. `annot=True` prints the numeric value in each cell. `fmt='.2f'` controls the decimal precision. `vmin/vmax` pins the colour scale so it always spans $[-1, 1]$ regardless of the data.

Example Code

```
num_cols = ['sepal length (cm)', 'sepal width (cm)',
            'petal length (cm)', 'petal width (cm)']
corr = iris_df[num_cols].corr()

fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            vmin=-1, vmax=1, linewidths=0.5, ax=ax)
ax.set_title('Feature correlation matrix -- Iris')
plt.tight_layout()
plt.show()
```

Exercise

1. Compute and display the correlation heatmap for all 13 numeric features of the Wine DataFrame. Which two features are most strongly positively correlated?

2. Add `mask=np.triu(np.ones_like(corr, dtype=bool))` to hide the upper triangle (which is redundant in a symmetric matrix) so only the lower triangle is shown.
3. Create a heatmap of the per-class mean of every feature: first call `.groupby('wine_class').mean()` to get a 3×13 summary table, then pass that to `sns.heatmap`.

2.6 Bar and Count Plots (barplot & countplot)

Concept

`sns.barplot()` estimates a statistic (default: mean) per category and draws error bars automatically – no manual numpy aggregation needed. `estimator` can be changed to 'median', 'sum', or any NumPy function. `sns.countplot()` is a convenience function that simply counts rows per category – the first thing to plot when checking class balance.

Example Code

```
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

# mean with 95% CI error bars
sns.barplot(data=iris_df, x='species', y='petal length (cm)',
            hue='species', palette='tab10', legend=False,
            estimator='mean', errorbar='sd', ax=axes[0])
axes[0].set_title('Mean petal length per species ($\pm$1 SD)')

# class counts
size_class = pd.cut(iris_df['sepal length (cm)'], bins=3,
                    labels=['small', 'medium', 'large'])
iris_df['size_class'] = size_class
sns.countplot(data=iris_df, x='size_class', hue='species',
              palette='tab10', ax=axes[1])
axes[1].set_title('Sepal size class counts by species')

plt.tight_layout()
plt.show()
```

Exercise

1. Use `sns.barplot` to compare the mean proline of each Wine class. Set `errorbar='sd'`.
2. Use `sns.countplot` to check class balance in the Wine dataset. Is it balanced?
3. Create a grouped bar chart of mean alcohol and mean color_intensity side-by-side per class. Hint: melt the DataFrame first: `pd.melt(wine_df, id_vars='wine_class', value_vars=['alcohol', 'color_intensity'])`, then pass the melted DataFrame to `barplot` with `x='wine_class', y='value', hue='variable'`.

2.7 FacetGrid — Small Multiples

Concept

`sns.FacetGrid()` divides a `DataFrame` into subsets by one or more columns and plots the same graph for each subset – a technique called **small multiples**. You specify the split column(s) with `col=` (horizontal) and `row=` (vertical), then call `.map()` with any Matplotlib or Seaborn plotting function. This is the fastest way to produce a full multi-class, multi-feature exploratory overview.

Example Code

```
g = sns.FacetGrid(iris_df, col='species', height=3.5, aspect=0.9)
g.map(sns.histplot, 'petal length (cm)', bins=12, kde=True, color='steelblue')
g.set_titles(col_template='{col_name}')
g.set_axis_labels('Petal length (cm)', 'Count')
g.figure.suptitle('Petal length distribution per species', y=1.02)
plt.tight_layout()
plt.show()
```

Exercise

1. Create a `FacetGrid` for the Wine `DataFrame` with `col='wine_class'` and plot a `histplot` of `alcohol` in each panel.
2. Change the plotting function to `sns.scatterplot` and map `color_intensity` on x and `alcohol` on y. Are the patterns consistent across classes or do the classes scatter differently?
3. Add a `row=` dimension by binning another feature into two groups (e.g. `high_phenols = pd.cut(wine_df['total_phenols'], bins=2, labels=['low', 'high'])`) and set `row='high_phenols'`, `col='wine_class'`. You should get a 2×3 grid of panels.

3 Capstone Exercise — A Complete Exploratory Visual Analysis

Exercise

Mini-Project: Visual EDA of the Wine Dataset. Using both Matplotlib and Seaborn, produce a complete Exploratory Data Analysis (EDA) figure set for the Wine dataset. Your notebook should contain **exactly the following eight figures** in order, each with a clear title and axis labels:

1. **Class balance:** a bar/count chart showing how many samples each `wine_class` has.
2. **Feature overview:** a 3×5 subplot grid of histograms, one per numeric feature, colour-coded by class (leave the 15th panel blank if needed).
3. **Scatter – best separator:** a Seaborn scatter plot of the two features you judge to be the best class separators, coloured and styled by class.
4. **Distribution comparison:** overlapping kdeplots of `proline` and `alcohol` (two panels side-by-side), one per feature, coloured by class.
5. **Box vs. violin:** a side-by-side box and violin plot of `color_intensity` per class.
6. **Grouped bar:** mean $\pm$ SD of `flavanoids`, `total_phenols`, and `proanthocyanins` per class as a grouped bar chart.
7. **Correlation heatmap:** lower-triangle correlation heatmap of all numeric features, with numeric annotations.
8. **Pairplot:** a Seaborn pair plot restricted to the four features you found most discriminating, coloured by class.

After generating all eight figures, write a short paragraph (4–6 sentences, as a Markdown cell) summarising what you learned about the dataset: which features best separate the classes, whether there is any class imbalance, and which pair of features has the strongest correlation.

Appendix: Quick Reference Cheat Sheet

Matplotlib

Call	Purpose
<code>fig, ax = plt.subplots(figsize=)</code>	Create a Figure + single Axes
<code>fig, axes = plt.subplots(r, c)</code>	Grid of $r \times c$ Axes panels
<code>axes.ravel()</code>	Flatten 2-D axes array for easy looping
<code>ax.plot(x, y)</code>	Line plot
<code>ax.scatter(x, y, c=, s=, alpha=)</code>	Scatter plot
<code>ax.hist(data, bins=, density=)</code>	Histogram
<code>ax.bar(x, height, yerr=, capsize=)</code>	Bar chart with optional error bars
<code>ax.boxplot(data, tick_labels=)</code>	Box plot (<code>patch_artist=True</code> for colour)
<code>ax.set_title/xlabel/ylabel()</code>	Panel labels
<code>ax.legend()</code>	Draw legend from labelled artists
<code>ax.axhline(y) / ax.axvline(x)</code>	Horizontal / vertical reference line
<code>ax.set_xlim() / ax.set_ylim()</code>	Fix axis ranges
<code>fig.suptitle()</code>	Shared title above all panels
<code>plt.tight_layout()</code>	Auto-adjust spacing
<code>plt.savefig(path, dpi=)</code>	Save to file

Seaborn

Call	Purpose
<code>sns.set_theme(style=)</code>	Set global theme (<code>whitegrid</code> , <code>ticks</code> , ...)
<code>sns.scatterplot(hue=, style=)</code>	Scatter with automatic class colouring
<code>sns.histplot(kde=True, bins=)</code>	Histogram with optional KDE overlay
<code>sns.kdeplot(fill=True, alpha=)</code>	Smooth density curve
<code>sns.boxplot(hue=, legend=False)</code>	Box plot per category
<code>sns.violinplot(inner='quartile')</code>	Violin plot with density + box
<code>sns.barplot(estimator=, errorbar=)</code>	Bar chart of a statistic per group
<code>sns.countplot(x=, hue=)</code>	Count plot (class balance check)
<code>sns.pairplot(diag_kind=, vars=)</code>	Full pairwise scatter matrix
<code>sns.heatmap(annot=True, cmap=)</code>	Colour-coded matrix (e.g. correlations)
<code>sns.FacetGrid(col=, row=)</code>	Small-multiples grid of panels
<code>g.map(func, col_name)</code>	Apply a plot function to each FacetGrid panel

End of Practical Session 3 Lab Sheet.